

Nama: Hans Malvin Djojo

<https://github.com/hansmalvin/LastMile>

Step-by-Step Instructions

Step 1: Creating a Custom Hook for Fetching Data

Create a new directory:

Inside the src directory, create a new directory named hooks.

```
bash
mkdir src/hooks
```

Create the useFetch hook:

Inside the hooks directory, create a new file named useFetch.js.

```
bash
touch src/hooks/useFetch.js
```

Implement the useFetch hook:

Open src/hooks/useFetch.js and add the following code:

```
javascript
import { useState, useEffect } from 'react';
const useFetch = (url) => {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchData = async () => {
      try {
        const response = await fetch(url);
        if (!response.ok) {
          throw new Error(HTTP error! Status: ${response.status});
        }
        const json = await response.json();
        setData(json);
        setLoading(false);
      } catch (err) {
        setError(err);
        setLoading(false);
      }
    }
  }, [url]);
}
```

```

}
};

fetchData();
}, [url]);

return { data, loading, error };
};

export default useFetch;

```

Explanation: This hook takes a URL as input, fetches data from the URL, and returns the data, loading state, and error state. It uses `useState` to manage the state and `useEffect` to perform the data fetching when the URL changes. The `try...catch` block handles potential errors during the fetch operation.

Step 2: Using the `useFetch` Hook in a Component

Modify `App.js`:

Open `src/App.js` and replace its content with the following code:

```

javascript
import React from 'react';
import useFetch from './hooks/useFetch';

function App() {
  const { data, loading, error } =
    useFetch('https://jsonplaceholder.typicode.com/todos');

  if (loading) return

  Loading...

  ;
  if (error) return
  Error: {error.message}

  ;
  return (

    Todos
    {data && data.map(todo => (
    {todo.title}
    ))}
  );
}

export default App;

```

Explanation: This component uses the useFetch hook to fetch a list of todos from a sample API. It displays a loading message while the data is being fetched, an error message if there is an error, and a list of todos once the data is loaded. This demonstrates how to consume a custom hook in a React component.

Step 3: Creating a Higher-Order Component (HOC)

Create a new directory:

Inside the src directory, create a new directory named hocs.

```
bash
mkdir src/hocs
```

Create the withLoading HOC:

Inside the hocs directory, create a new file named withLoading.js.

```
bash
touch src/hocs/withLoading.js
```

Implement the withLoading HOC:

Open src/hocs/withLoading.js and add the following code:

```
javascript
import React from 'react';
const withLoading = (WrappedComponent) => {
  return function WithLoadingComponent({ isLoading, ...props }) {
    if (isLoading) {
      return
      Loading...
    }
    ;
  }
  return ;
};
};
export default withLoading;
```

Explanation: This HOC takes a component as input and returns a new component that displays a loading message while the `isLoading` prop is true. Otherwise, it renders the original component with all the original props. This allows you to add loading indicators to any component without modifying its original code.

Step 4: Using the `withLoading` HOC

Create a new component:

Inside the `src` directory, create a new file named `TodoList.js`.

```
bash
touch src/TodoList.js
```

Implement the `TodoList` component:

Open `src/TodoList.js` and add the following code:

```
javascript
import React from 'react';
const TodoList = ({ todos }) => {
  return (
    {todos && todos.map(todo => (
      {todo.title}
    ))}
  );
};
export default TodoList;
```

Explanation: This component simply renders a list of todos. Modify `App.js` to use the HOC:

Open `src/App.js` and modify it to use the `withLoading` HOC:

```
javascript
import React from 'react';
import useFetch from '../hooks/useFetch';
import withLoading from '../hocs/withLoading';
import TodoList from '../TodoList';
const EnhancedTodoList = withLoading(TodoList);

function App() {
```

```

const { data, loading, error } =
useFetch('https://jsonplaceholder.typicode.com/todos');

if (error) return

Error: {error.message}

;
return (

Todos

);
}
export default App;

```

Explanation: We import the withLoading HOC and the TodoList component. We then create EnhancedTodoList which is the TodoList component wrapped with the withLoading HOC. We pass the loading state from the useFetch hook as the isLoading prop to the EnhancedTodoList. This allows the HOC to handle the display of the loading indicator.

Step 5: Creating a Component with Render Props

Create a new directory:

Inside the src directory, create a new directory named renderprops.

```

bash
mkdir src/renderprops

```

Create the MouseTracker component:

Inside the renderprops directory, create a new file named MouseTracker.js.

```

bash
touch src/renderprops/MouseTracker.js

```

Implement the MouseTracker component:

Open src/renderprops/MouseTracker.js and add the following code:

```

javascript
import React, { useState, useEffect } from 'react';

```

```

const MouseTracker = ({ render }) => {
const [mousePosition, setMousePosition] = useState({ x: 0, y: 0 });

useEffect(() => {
const handleMouseMove = (event) => {
setMousePosition({ x: event.clientX, y: event.clientY });
};

window.addEventListener('mousemove', handleMouseMove);

return () => {
window.removeEventListener('mousemove', handleMouseMove);
};
}, []);

return render(mousePosition);
};

export default MouseTracker;

```

Explanation: This component tracks the mouse position and calls the render prop with the current mouse position. The render prop is a function that returns a React element. This allows you to customize the UI based on the mouse position without modifying the MouseTracker component.

Step 6: Using the MouseTracker Component

Modify App.js to use the MouseTracker component:

Open src/App.js and modify it to use the MouseTracker component:

```

javascript
import React from 'react';
import useFetch from './hooks/useFetch';
import withLoading from './hocs/withLoading';
import TodoList from './TodoList';
import MouseTracker from './renderprops/MouseTracker';
const EnhancedTodoList = withLoading(TodoList);

function App() {
const { data, loading, error } =
useFetch('https://jsonplaceholder.typicode.com/todos');

if (error) return

Error: {error.message}

;

```

```
return (

Todos

Mouse Tracker

render={mousePosition => (
Mouse position: X: {mousePosition.x}, Y: {mousePosition.y}
)}
/>
);
}
export default App;
```

answer:

